

IT LAB

Name: Subhrajit Naskar

Roll: 002310501004

BCSE – 3

Assignment -2

Photo Sharing Application

Introduction:

This project implements an asynchronous, multi-client photo-sharing platform built with Node.js and the WebSocket (ws) library. Its primary goal is to offer a real-time environment where several users can connect, interact, and exchange images seamlessly. Through WebSocket communication, a persistent connection is maintained between the server and each client, eliminating the need for repeated request–response cycles and enabling instant data transfer.

The application allows users to join designated groups, upload photos, and add comments to shared images. This group-based approach keeps conversations organized and ensures that updates are visible only to members of the relevant group. Whenever a user uploads an image or posts a comment, the server instantly broadcasts the update to all other connected users within that group.

Using WebSocket technology provides fast, bi-directional communication, resulting in lowlatency updates and smoother interactions. This not only reduces unnecessary network traffic but also enhances the overall user experience. Additionally, Node.js's non-blocking, event-driven architecture enables efficient handling of multiple concurrent users, making the system scalable, responsive, and well-suited for real-time collaborative applications.

Technologies Used:

- **Node.js**

Serves as the server-side runtime environment, enabling the creation of a fast, scalable, and non-blocking backend using JavaScript.

- **Express.js**

A lightweight Node.js framework used for managing routes, middleware, and handling standard HTTP requests efficiently.

- **WebSocket (ws)**

Provides real-time, bi-directional communication between the server and clients, allowing instant photo sharing, messaging, and updates without page reloads.

- **Pug Template Engine**

Used to generate dynamic HTML content on the server side, offering a clean, simple, and readable syntax for building UI templates.

- **HTML, CSS, and JavaScript**

Form the core technologies for the client-side interface—structuring the webpage, styling the layout, and implementing interactive features.

System Architecture

The application follows a client–server model, where a central server coordinates all interactions and data flow between connected users. The server manages WebSocket connections, tracks active user sessions, handles group assignments, stores uploaded images, and forwards messages or updates to the correct group members.

Users interact with the system through a web browser, which establishes a WebSocket connection to the server. After connecting, each user joins a specific group by entering a

username and group ID. Once part of a group, they can upload images, view media shared by others, and send comments—all of which are delivered instantly through the real-time communication channel.

This architecture promotes smooth and efficient data exchange while keeping system logic well-organized. The clear separation between client-side and server-side operations enhances scalability, reduces maintenance complexity, and ensures consistent performance as the number of users grows.

Communication in Node.js:

Node.js operates on an event-driven, non-blocking I/O architecture, allowing it to manage multiple tasks at the same time without waiting for one operation to finish before starting another. This makes it highly suitable for real-time applications.

Every WebSocket connection in the system is handled asynchronously, enabling the server to interact with numerous clients at once. Rather than creating a separate thread for each connection, Node.js relies on a single-threaded event loop that efficiently processes incoming events, resulting in lower overhead and better performance.

The server listens for key WebSocket events such as *connection*, *message*, and *close*. When a client connects, the server registers the session and links it to the appropriate group. When a message or image is received, the server immediately processes the data and broadcasts it to all members of that group, ensuring real-time updates.

Implementation

```
// Middleware
```

```
app.use(express.static('public')); app.use(express.json());
```

```
// Data structures to manage users, groups, and photos
const users = new Map(); // socketId -> {username, groups: []}
const groups = new Map(); // groupId -> {name, members: [], photos: []}
```

```
const photos = new Map(); // photoId -> {data, uploadedBy, timestamp, comments: []}
```

```
// Generate unique IDs
```

```
const generateId = () => Math.random().toString(36).substr(2, 9);
```

```

// WebSocket connection handling io.on('connection',
(socket) => {
    console.log(`New client connected: ${socket.id}`);

    // User registration
    socket.on('register', (username) => {
users.set(socket.id, {          id:
socket.id,          username:
username,          groups: []
    });
    socket.emit('registered', {
userId: socket.id,          username:
username
    });

    // Broadcast updated user list to all clients
    broadcastUserList();

    console.log(`User registered: ${username} (${socket.id})`);
});

// Create a new group
    socket.on('create-group', (data) => {
const { groupName, memberIds } = data;
const groupId = generateId();

        // Add creator to members if not already included
        if (!memberIds.includes(socket.id)) {
            memberIds.push(socket.id);
        }

        const group = {
id: groupId,          name:
groupName,          members:
memberIds,          photos:
[],          createdBy:
socket.id,
            createdAt: new Date().toISOString()
        };
        groups.set(groupId, group);

        // Add group to each member's group list
        memberIds.forEach(memberId => {
const user = users.get(memberId);
            if (user) {
                user.groups.push(groupId);
            }
        });
    });
}

```

```

        }
    });

    // Notify all group members
    memberIds.forEach(memberId => {
    io.to(memberId).emit('group-created', {
        groupId: groupId,
        groupName: groupName,
        members: memberIds.map(id => users.get(id)?.username
    || 'Unknown'),
        createdBy: users.get(socket.id)?.username
    });
    });

    console.log(`Group created: ${groupName} (${groupId}) by
    ${socket.id}`);
    });

    // Share photo with everyone (public)
    socket.on('share-photo-public', (data) => {
    const { photoData, caption } = data;
    const photoId = generateId();          const
    user = users.get(socket.id);

    const photo = {
    id: photoId,
    data: photoData,
    caption: caption,
        uploadedBy: user?.username || 'Anonymous',
        uploaderId: socket.id,
        timestamp: new Date().toISOString(),
        comments: [],
    type: 'public'
    };
    photos.set(photoId, photo);

    // Broadcast to all connected users
    io.emit('new-photo', photo);

    console.log(`Public photo shared by ${user?.username}
    (${photoId})`);
    });

    // Share photo with a specific group
    socket.on('share-photo-group', (data) => {
    const { photoData, caption, groupId } = data;
    const group = groups.get(groupId);      const user
    = users.get(socket.id);

```

```

        if (!group) {
            socket.emit('error', { message: 'Group not found' });
        }
        return;
    }
    if (!group.members.includes(socket.id)) {
        socket.emit('error', { message: 'You are not a member of
this group' });
        return;
    }
    const photoId = generateId();
    const photo = {
        id: photoId,
        data: photoData,
        caption: caption,
        uploadedBy: user?.username || 'Anonymous',
        uploaderId: socket.id,
        timestamp: new Date().toISOString(),
        comments: [],
        type: 'group',
        groupId: groupId,
        groupName: group.name
    };
    photos.set(photoId, photo);
    group.photos.push(photoId);

    // Send to all group members
    group.members.forEach(memberId => {
        io.to(memberId).emit('new-photo', photo);
    });

    console.log(`Group photo shared in ${group.name} by
${user?.username} (${photoId})`);
});

// Add comment to a photo
socket.on('add-comment', (data) => {
    const { photoId, comment } = data;
    const photo = photos.get(photoId);
    const user = users.get(socket.id);

    if (!photo) {
        socket.emit('error', { message: 'Photo not found' });
        return;
    }

    // Check if user has access to this photo
    if (photo.type === 'group') {

```

```

        const group = groups.get(photo.groupId);
if (!group || !group.members.includes(socket.id)) {
socket.emit('error', { message: 'You do not have access to this
photo' });
return;
}
}
const commentObj = {
id: generateId(),
text: comment,
username: user?.username || 'Anonymous',
userId: socket.id,
timestamp: new Date().toISOString()
};
photo.comments.push(commentObj);

// Notify relevant users about the new comment
if (photo.type === 'public') {
io.emit('new-comment', { photoId:
photoId, comment: commentObj
});
} else if (photo.type === 'group') {
const group = groups.get(photo.groupId);
group.members.forEach(memberId => {
io.to(memberId).emit('new-comment', {
photoId: photoId, comment:
commentObj
});
});
}
console.log(`Comment added to photo ${photoId} by
${user?.username}`);
});

// Get user's groups
socket.on('get-groups', () => {
const user = users.get(socket.id);
if (!user) return;

const userGroups = user.groups.map(groupId => {
const group = groups.get(groupId);
if
(!group) return null;
return {
id: group.id,
name: group.name,
members: group.members.map(id => {
const member = users.get(id);
return member ? member.username : 'Unknown';

```

```

        })),
        memberIds: group.members,
photoCount: group.photos.length,
        createdBy: users.get(group.createdBy)?.username ||
'Unknown'
    });
    }).filter(g => g !== null);

    socket.emit('groups-list', userGroups);
  });

  // Get all users (for group creation)    socket.on('get-
users', () => {
    const userList = Array.from(users.values())
    .filter(user => user.id !== socket.id)
    .map(user => ({
id: user.id,
        username: user.username
    }));
    socket.emit('users-list', userList);
  });

```

client.js

```

function joinGroup() {
  ws = new WebSocket(`ws://${location.host}`);

  ws.onopen = () => {
const payload = {
  type: "JOIN",
  username: username,
  group: group    };
    ws.send(JSON.stringify(payload));
  };
}

function uploadPhoto() {
  if (!file) return;
  ws.send(file);    // send raw file data
}

function sendComment(photoUrl) {
  const commentPayload = {
  type: "NEW_COMMENT",
  url: photoUrl,      text:
input.value.trim()

```



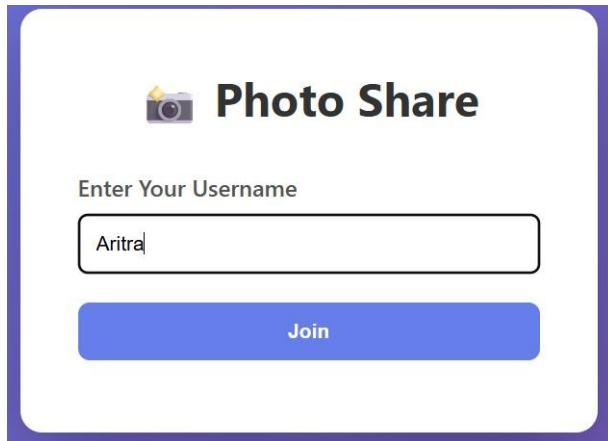
```

    };
    ws.send(JSON.stringify(commentPayload));
    input.value = ""; // clear the comment box
}

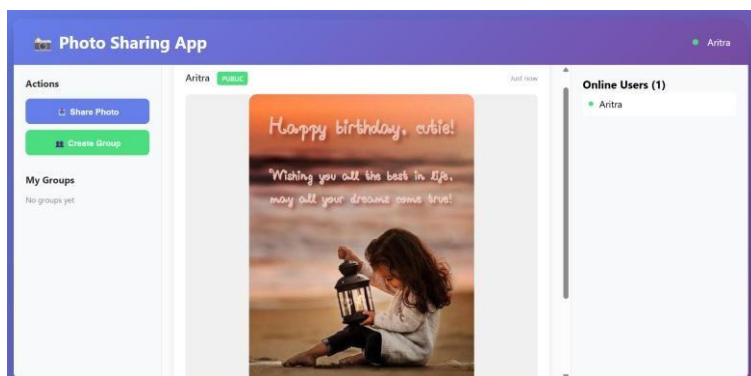
```

OUTPUT:

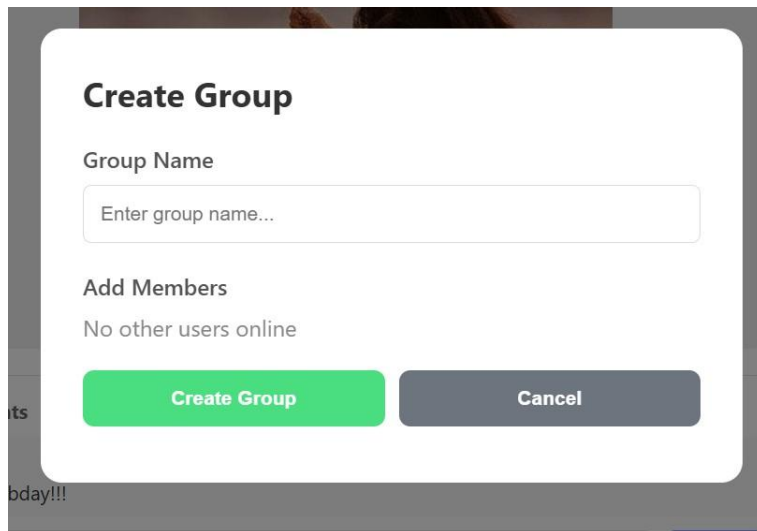
Login page:



Main Window



Group Creation:



Create Group

Group Name

Enter group name...

Add Members

No other users online

Create Group Cancel

Protocol Layers

- **Application Layer – WebSocket, HTTP**
The browser first connects using HTTP, which is then upgraded to a WebSocket channel to enable continuous, real-time communication.
- **Transport Layer – TCP**
Provides reliable and ordered delivery of data, ensuring that messages and images reach the destination correctly.
- **Network Layer – IP**
Responsible for addressing and routing packets between devices across different networks.
- **Data Link Layer – Ethernet / Wi-Fi**
Handles frame transmission, MAC addressing, and communication within the local network.
- **Physical Layer – Transmission Medium**
Consists of the actual hardware such as cables, wireless signals, routers, and network interfaces used to transmit data.

7. Observations

- Real-time behaviour is achieved through long-lived WebSocket connections.
- The Node.js event-driven model efficiently manages a large number of simultaneous users.
- Broadcasting within groups keeps communication structured and accessible only to permitted members.

- Even under heavy usage, the application remains smooth and responsive due to asynchronous processing.

8. Conclusion

This WebSocket-powered photo-sharing system showcases effective real-time communication between multiple clients. The use of persistent WebSocket channels allows instant delivery of photos and comments, creating a highly interactive user experience. Node.js further enhances performance with its non-blocking, event-driven architecture, enabling the server to scale and handle numerous concurrent users efficiently. Group-focused communication adds an extra layer of organization and security by limiting data access to authorized participants. Overall, this project offers valuable hands-on understanding of building scalable, responsive, real-time web applications.